Veritas DFIR: Verifiable Evidence Handling and Validated Artifact Parsing for Digital Forensics

lumidren, November 2026

Abstract

Digital forensic conclusions are only as strong as two things a court can test: whether the evidence examined is the evidence seized, and whether the tools that interpreted it are correct. Veritas DFIR is an open platform built around both. Evidence is hashed with three algorithms at acquisition, stored write-once under its SHA-256, and every subsequent access, extraction, parse, export and report is recorded in an Ed25519-signed, hash-chained custody ledger that an independent verifier re-checks from scratch. Every parser is validated field by field against a reference implementation on public test data; across 68 parsers the platform agrees with The Sleuth Kit, libyal, Plaso, dissect, scapy and LIEF on 99.41% of 1,483,226 field comparisons, and every disagreement is explained. Parsers are memory-safe Rust or exception-contained Python, run in resource-capped sandboxed workers, and are fuzzed; fuzzing found and we fixed an integer overflow in the systemd-journal parser. On a real 4.87 GB intrusion image (NIST CFReDS *Hacking Case*) the pipeline runs end to end in 134 seconds on one worker and reconstructs the intruder's toolkit with execution proven by independent artifact sources. Reports regenerated from the same case state are byte-identical. This paper describes the design, the validation method and results, the custody ledger and its verification, measured performance, alignment with ISO/IEC 27037, NIST SP 800-86, SWGDE and ACPO guidance, and the limitations an examiner should disclose.

1. Introduction

Commercial and open-source forensic suites are trusted largely by reputation. Examiners can state which tool and version they used, but they rarely can show, for the specific artifact types in a case, how often that tool's interpretation agrees with an independent implementation, nor can they hand opposing counsel a machine-checkable record of every time the evidence was touched. Daubert-style scrutiny asks precisely these questions: is the technique testable, has it been tested, what is its error rate, and are there standards controlling its operation.

Veritas DFIR treats those questions as engineering requirements:

1. **Integrity is cryptographic, not procedural.** Each evidence object is addressed by its hash and each handling event is a signed ledger entry linked to its predecessor.
2. **Correctness is measured.** Each parser ships with a parity test that compares its output with a reference implementation on public data and records per-field agreement. Continuous integration rejects any change that lowers agreement.
3. **Hostile input is assumed.** The subject of an investigation controls the bytes being parsed; parsers are memory-safe, bounded and isolated.
4. **Conclusions are human.** Analytics propose findings; an examiner confirms them with a rationale and a different reviewer approves them before they can appear in a report.
5. **Outputs are reproducible.** Parsers and reports are deterministic, so a second examiner on a second machine obtains identical bytes.

The remainder of the paper follows the platform from acquisition to court.

1.1 Related work

The Sleuth Kit and Autopsy established open, scriptable file-system analysis; Plaso (log2timeline) established the super-timeline and a shared event vocabulary that Veritas adopts; the libyal libraries provide carefully documented format implementations; Volatility defines memory analysis; dissect offers a second modern

implementation of many Windows formats. Each is a reference in our validation precisely because it is independent of our code. Prior validation efforts such as the NIST Computer Forensics Tool Testing programme test tools against specifications for selected functions and publish reports per tool version. Veritas differs in three ways: validation is continuous and per field rather than per release and per function; the comparison runs against several independent implementations at once, which lets a disagreement be attributed to a specific side; and custody is part of the data model rather than a separate case-management concern. Hash-chained logs and signed audit trails are well known from certificate transparency and secure logging; Veritas applies them to evidence handling with an offline verifier aimed at a court-appointed expert.

2. Architecture

Veritas has a Rust core and a Python service layer (Figure 1). The Rust workspace contains the hashing, ledger, store, Expert Witness Format (EWF) and file-system crates, the artifact parsers, the acquisition agent and two command-line tools: `veritas`, an umbrella tool, and `veritas-verify`, the independent custody verifier. The Python package contains the HTTP API, the processing pipeline, the SQLite- or ClickHouse-backed timeline, analytics, reporting, and parsers whose natural implementation is Python (SQLite- and ESE-based artifacts, cloud logs, mobile backups, Volatility 3 integration). A Next.js examiner application sits on top of the API.

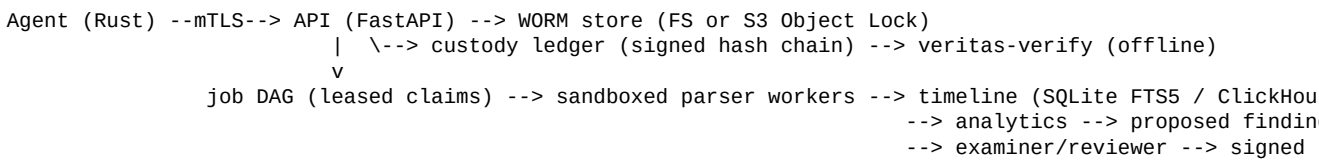

Figure 1. Data flow. Every arrow that reads evidence bytes appends a custody entry.

Two deployment profiles share the code. The native profile runs on one workstation with SQLite and the local file system and needs no containers; it is how the quickstart and all measurements in this paper were run. The cluster profile uses Postgres, ClickHouse and S3 with Object Lock on Kubernetes, with workers scaled on queue depth.

All events share one schema: Plaso's `timestamp_desc`, `data_type`, `message`, `source_short` and `source_long`, extended with provenance (`evidence_sha256`, the path inside the evidence, the byte offset or record index, parser name and version) and an explicit clock-skew correction. Timestamps are integer nanoseconds in UTC; floating point never appears in a signed or hashed structure.

3. Acquisition and evidence integrity

3.1 Acquisition

The agent images a device or file to raw segments or to EnCase 6 E01, hashing the stream in a single pass with MD5, SHA-256 and SHA3-256 so that the source is read once. Before imaging it checks write protection through the operating system (disk read-only attributes on Windows, `/sys/block/*/ro` and `blockdev --getro` on Linux, `diskutil` on macOS) and records the result. Unreadable blocks are re-read sector by sector; sectors that still fail are zero-filled and listed in the image's `error2` section and in the acquisition record. The agent measures the host clock against NTP and records the offset and its uncertainty. It then appends a signed `acquire` entry carrying the device description, hashes, the agent binary's SHA-256 and the full command line.

Live triage collects volatile and persistent artifacts into a ZIP bundle whose manifest lists every file with its hash; unprivileged runs record each refusal instead of failing. On Windows the agent reads `$MFT`, `$LogFile` and `$UsnJrnl:$J` directly from the volume.

The EWF writer was validated against libewf: for eight images, including our writer's output read by libewf and libewf-produced images read by our reader, media size, MD5, SHA-1, header values and every media byte agree.

3.2 Write-once, encrypted storage

The store addresses each object by the SHA-256 of its plaintext. The local backend commits through a temporary file, `fsync` and rename, then marks the object read-only and refuses to replace an existing address. The S3 backend refuses to start unless the bucket has Object Lock enabled and writes every object in compliance mode. Objects are encrypted in 1 MiB frames with XChaCha20-Poly1305 under a key derived per case from a master key held in KMS; the frame index is bound into the nonce and the case identifier into the associated data, so frames cannot be reordered, moved between objects, or decrypted under another case's key. Multi-segment images are stored as one object per segment, all listed in the ingest entry.

4. The custody ledger

4.1 Entry structure

A ledger is a sequence of entries. Each entry body contains the ledger identifier, a dense sequence number, the SHA-256 of the previous entry, an RFC 3339 timestamp, the actor and the identifier of the actor's key, the entry kind (genesis, acquire, ingest, access, export, transform, note, key enrolment, key revocation, anchor), the evidence identifier, hashes, device and time-source records, the tool's name, version, binary hash and command line, a purpose and structured details.

The body is serialised to canonical JSON: object keys sorted by UTF-16 code units, no insignificant whitespace, integers only, minimal escaping. The actor signs the canonical bytes with Ed25519, and the entry hash is `SHA-256(canonical(body) || signature)`, which becomes the next entry's predecessor hash. Signing the body and chaining over body and signature means that neither content nor signature can change without breaking every later link.

The access entry the pipeline writes when it parses a prefetch file has this shape (an illustrative entry generated with a test key; hashes abbreviated; the signature and entry hash cover the full body):


```
{"ledger_version": 1, "ledger_id": "L-C-20261104-FB7C41", "seq": 1,
 "prev_hash": "58fe3125...f53673", "timestamp": "2026-11-04T14:59:17.898995Z",
 "actor": "veritas-pipeline", "actor_key_id": "10ba682c8ad13513971e8b56881aab8b",
 "kind": "access", "evidence_id": "EV-5A1C0E9B2F", "hashes": null, "device": null,
 "time_source": null,
 "tool": {"name": "veritas-api", "version": "0.1.0", "binary_sha256": null, "command_line": "veritas-api"}
 "purpose": "parse with prefetch",
 "details": {"logical_path": "/WINDOWS/Prefetch/CAIN.EXE-23D61279.pf", "object_sha256": "...",
             "parser": "prefetch", "parser_version": "0.1.0", "job_id": "J-3c9f0a71d2e4"},
 "signature": "a8fb0c9f...7176a009", "entry_hash": "b3ab61be...2dbd1a3"}
```


Examiner keys are held server-side, sealed with a key derived from the master key, and never logged; a test drives a full API session at debug log level and scans the captured logs, the audit trail and the ledger for passwords, bearer tokens, the master key and every signing seed.

The API writes entries from Python and the agent from Rust. Both produce byte-identical canonical forms; a cross-language test writes a ledger in Python and verifies it with the Rust verifier, and flips one byte to confirm rejection. A subtle case is key ordering: sorting by UTF-16 code units places U+1F600 (surrogate U+D83D) before U+FF61, the reverse of code-point order, and the test pins it.

4.2 What is recorded

Every evidence read is an entry. Ingestion records the stored hashes; each parse records the parser and version; each hex-viewer read records the offset and length; each artifact extracted from an image records its hash and logical path as a `transform`; exports and reports record their own hashes. On the *Hacking Case* run, processing one image produced 263 entries, every one of them signed.

4.3 Independent verification

`veritas-verify` takes either a raw ledger with a key registry or a signed custody export, and checks: the sequence is dense and starts with a single genesis entry; each predecessor hash and entry hash recomputes; timestamps never decrease; each signature verifies under a key that was enrolled, inside its validity window and not revoked at that time; key-management entries are signed by an administrative key; anchors match the chain

at their sequence numbers; and, for exports, the outer signature, the stated head and the entry count. It prints PASS or FAIL with every failing check.

Tests cover modification of any field, deletion, reordering, truncation of the tail (caught by the signed head), substitution by a re-signed entry from an unenrolled key, and tampering with an export on disk. Periodic anchoring writes the chain head to an external append-only sink, bounding how long an insider holding a valid key and root access could rewrite recent history undetected.

5. Parsers and their validation

5.1 Coverage

Parsers cover partition tables, NTFS (including alternate data streams, `$I30` slack, the USN journal and `$LogFile` records), FAT and exFAT, ext2/3/4, APFS, Volume Shadow Copies, carving, and BitLocker, LUKS and APFS decryption; Windows registry hives with transaction-log replay and deleted-cell recovery and fifteen registry artifact plugins; EVTX with a complete binary-XML engine and legacy EVT; prefetch through Windows 11 with an LZXPRESS Huffman decoder; LNK, shell items, OLE compound files and jump lists; systemd journals; plists; scheduled tasks; 58 Python parser classes for browsers, SRUM, email, Office documents and Linux and macOS logs and databases; memory analysis through Volatility 3; packet captures; executables; six cloud audit-log formats; and iOS and Android backups. Section-by-section coverage and exclusions are listed in the parser documentation.

5.2 Validation method

For every parser we chose a reference implementation that is independent of ours and widely used in casework: libyal bindings (libevtx, libregf, libscca, liblnk, libfwsi, libolecf, libfsntfs, libfsext, libfsfat, libfsapfs, libvshadow, libewf), The Sleuth Kit via pytsk3, dissect, Plaso run in-process, scapy and LIEF. Where no reference exists on the test platform we wrote a small independent decoder from the published format description (journald, ShimCache, UserAssist) and say so in the result.

Test data is public and pinned: 136 files from the Plaso and dfVFS test suites and NIST CFReDS, each verified by SHA-256 before use. For each record present in both outputs, each selected field is normalised (timestamps to microseconds, paths to one separator convention) and compared. A result file records, per field, how many records agreed out of how many were compared, plus notes for every disagreement. A report generator aggregates the files, and a gate fails CI if any field's ratio falls below the committed baseline.

5.3 Results

Family	Parsers	Agreeing comparisons	Agreement
File systems and volumes	10	199,713 / 199,755	99.98%
Windows, Linux and macOS artifacts (Rust)	9	1,206,340 / 1,215,014	99.29%
Python artifact parsers	46	65,374 / 65,421	99.93%
EWF, PCAP, PE	3	3,036 / 3,036	100.00%
All	**68**	**1,474,463 / 1,483,226**	**99.41%**

Table 1. Field-level agreement with reference implementations.

The disagreements are concentrated and explained:

- **Registry values (8,509 comparisons).** libregf returns the wrong half of the 4-byte inline data field for values shorter than four bytes and truncates large strings at the first NUL; dissect.regf agrees with Veritas byte for byte on all 165,270 keys of 11 hives.
- **FAT dates (14).** pyfsfat reports dates one month late; pytsk3 and a manual decode agree with us.
- **journald messages (165).** These payloads are XZ-compressed, which our reader reports as unsupported rather than decoding.

- **Python parsers (47).** Plaso drops two orphaned Firefox bookmarks and one multi-line sudo record, emits unset 2001-01-01 Santa timestamps that we omit, and differs by one rounding step on a handful of MSIE cache times.
- **EVTX record count.** In addition to the field results, Veritas keeps 89 records from one chunk whose checksum fails and flags them; libevtx drops them. An examiner sees the records and the flag.

5.4 Robustness

Rust parsers forbid `unsafe`, read through bounds-checked helpers, and cap recursion depth, entry counts and decompression output. Python parsers convert record-level failures to counted warnings and only raise a typed parser error. Property-based tests mutate and truncate every corpus sample; 14 cargo-fuzz targets seeded from the corpus executed about 5.5 million inputs in a 60-second smoke run per target. Fuzzing found one defect: a crafted systemd journal whose entry-array offset lay near the maximum machine word made an offset addition overflow and panic. The fix rejects out-of-range offsets before any arithmetic, and a regression test reproduces the original panic without the fix.

Parsers run in child processes with a wall-clock limit, a memory cap (a Windows job object or `RLIMIT_AS`), and no network: sockets are disabled in the child before any parser loads and, on Linux, the child runs in its own network namespace. Tests confirm that a hanging parser, a parser allocating a gigabyte, a parser opening a connection and a parser raising on a malformed record each fail only their own job.

5.5 Determinism

Twenty-one corpus samples spanning every parser family were parsed twice in fresh processes with different hash seeds and working directories; all 21 produced byte-identical output. CI repeats the check on Linux and Windows and compares digests across the two.

6. Analysis

The timeline store indexes every event with full-text search over messages and attributes, and covering indexes on time, type, host, user and evidence; every query is scoped by case in the data layer. Rules then propose findings:

- **Execution correlation** joins prefetch, Amcache, ShimCache, BAM, UserAssist, Security 4688 and Sysmon 1 per program and states which sources prove execution and which only prove presence. Known offensive and dual-use tools are matched by family across spellings (an executable, its installer, a Start-menu shortcut and a prefetch name truncated at 29 characters are one finding).
- **Timestomping** is graded. Installers set timestamps through the same API as timestomping tools, so on the Plaso reference Windows XP volume 4,294 of 13,064 files show the classic pattern of `$STANDARD_INFORMATION` creation time earlier than `$FILE_NAME` creation time with whole seconds. Veritas raises an individual high-severity finding only when the USN journal records a data change after the claimed modification time, a medium finding when the pattern is isolated within its directory outside installation trees, and otherwise a single low-severity summary. On that volume the result is one summary instead of thousands of accusations.
- **Lateral movement** builds a host graph from logons, explicit credentials, remote service installation, scheduled tasks, WMI, RDP and administrative share access.
- **Persistence, anti-forensics, user activity, file provenance** and **IOC sweeps** complete the set.

Findings remain proposals until an examiner confirms them with a written rationale and a reviewer who did not confirm them approves them. Re-running rules replaces only proposals nobody has acted on.

6.1 Casework, review and audit

Cases have members with one of four roles (viewer, examiner, reviewer, lead); a global administrator manages users and keys. Every case route resolves membership through one shared dependency, and a non-member

receives "not found" rather than "forbidden", so case identifiers cannot be probed. The findings state machine is enforced in the API: an examiner confirms or rejects with a rationale of at least a sentence; a reviewer approves or returns; the confirming examiner can never approve; only approved findings are reported. Bookmarks, notes, hypotheses with supporting and refuting events, tasks and saved views support collaboration. Every authenticated request is appended to a separate hash-chained audit log with secrets scrubbed, which answers "who looked at what, when" and is exported with the case. Personally identifiable information is masked by default with keyed, stable tokens, so examiners can still correlate identical values; unmasking requires a permission and is itself audited.

6.2 Processing pipeline

Each evidence item is processed by a directed acyclic graph of steps (verify, file-system timeline, artifact harvest, per-artifact parsing, analytics). A step's identity is the hash of the evidence, step name, parser version and parameters; a completed step is never recomputed for the same identity, and a crashed run resumes at the first incomplete step. Workers claim jobs with an atomic compare-and-set on a lease that is renewed with every progress report, so any number of workers can share one queue and a job abandoned by a dead worker is picked up after its lease expires. Disk images are harvested for known artifact locations on Windows (including XP and Server 2003 layouts), Linux and macOS; each extracted artifact becomes a stored, hashed, signed derived object before it is parsed.

7. Reporting

A report is a pure function of case state: approved findings, their supporting events, the evidence inventory and the custody chain, tool versions and binary hashes, and parity figures. The model has no clock reads and no unordered collections; its canonical hash is the report's state hash. The PDF is rendered with fixed metadata and a content-derived document identifier; the DOCX is re-packed with fixed entry order and timestamps. Generating twice from the same state produces identical bytes. Each cited artifact names its evidence hash, path, offset and the ledger entry that ingested the evidence. The report is signed with the examiner's key and recorded in the ledger.

8. Evaluation on an intrusion image

We processed the NIST CFReDS *Hacking Case* image, a Dell Latitude CPi laptop seized in an investigation of wireless traffic interception, through the full pipeline on a laptop with an Intel Core i7-12700H, 16 GB RAM and an NVMe SSD, using the native profile and one worker.

Step	Seconds
Register and store (two EWF segments, triple hash, custody)	9.5
Re-verify stored segments	7.6
File-system timeline (225,980 events)	35.7
Harvest and parse 126 artifacts in sandboxed workers	71.0
Analytics over 382,643 events	10.2
Total for 4.87 GB of media	**134**

Table 2. End-to-end processing, measured.

The harvest recovered the registry hives, user profile, Internet Explorer history, legacy event logs, recent-document shortcuts and Recycle Bin of the user "Mr. Evil". The rules proposed Cain, Ethereal, NetStumbler, LookAtLAN, a password-recovery tool and the Anonymizer as high- or medium-severity dual-use tool findings, each with execution proven by at least two of prefetch, ShimCache and UserAssist. Full-text searches over the case returned in 73 to 242 ms. The exported ledger verified with `veritas-verify`.

Per-stage measurements on synthetic data complete the picture: single-pass triple hashing at 755 GB/h, E01 imaging with fast compression at 421 GB/h, MFT parsing at 112,000 events per second, and SQLite timeline

ingest at 28,000 events per second with median search latency of 66 ms at one million events. A 500 GB image was not processed on the development machine; from these stages we model about one hour on one worker plus artifact parsing, which scales with the number of artifacts rather than the image size and runs in parallel workers.

8.1 Operations

The cluster profile ships as a Helm chart (API, workers autoscaled on queue depth through KEDA, web, pod disruption budgets, default-deny network policies, TLS ingress, non-root read-only containers with an optional gVisor runtime for workers) and Terraform for AWS (private networking, EKS, encrypted Postgres with point-in-time recovery, an Object Lock evidence bucket under KMS, WAF, CloudWatch and AWS Backup with vault lock). CI runs formatting, linting, type checks, the unit, parity, determinism, integration and browser suites, fuzz smoke tests, Semgrep, Trivy, cargo-audit, pip-audit and npm audit, and fails on high or critical findings; releases are signed with cosign and code-signing certificates and ship a CycloneDX SBOM. Grafana dashboards and Prometheus alerts cover throughput, parser errors, queue depth, search latency, storage growth and custody verification. The backup tool archives the databases, custody ledgers, anchors and key registry with a hash manifest, and the restore tool re-verifies every ledger chain with both verifiers, the audit chain and every stored object hash before declaring success; a drill on the native profile completed a backup in 1.1 s and a verified restore in 1.6 s for a test case.

9. Standards alignment

Requirement	Control in Veritas
ISO/IEC 27037 auditability, repeatability, reproducibility	Signed ledger with independent verifier; determinism tests; parity against independent tools
ISO/IEC 27037 preservation and documented handling	Write-block check, triple hashing, WORM store, per-access custody entries
NIST SP 800-86 collection, examination, analysis, reporting	Agent and intake; validated parsers; correlation rules; report template with limitations
SWGDE acquisition practice	Tool version and hash in every entry; acquisition hashes in E01 and ledger; bad-sector maps
ACPO principles 1 to 3	No modification of originals; actor and purpose recorded for live actions; complete audit trail
ACPO principle 4	Supported by roles and two-person review; competence remains procedural

Table 3. Summary of the standards matrix; the full mapping is in the compliance document.

10. Threat model summary

The threat model covers evidence tampering by insiders and operators, custody forgery, malicious evidence aimed at parsers, cross-case leakage, privilege abuse, key compromise and the supply chain. Each mitigation names the file and line implementing it, and a CI check fails if a referenced line no longer carries its marker. Residual risks are stated: an operator holding an unlocked examiner key and root on the ledger host can forge that examiner's entries until the next external anchor; memory acquisition on a compromised kernel can be subverted; and the verifier trusts the key registry it is given, which must be distributed out of band.

11. Limitations

- **500 GB throughput** is modelled from measured stages and one measured 4.87 GB run, not measured end to end.
- **Memory parity** against Volatility requires an examiner-supplied memory image; no public image is small enough to ship with the test corpus.
- **Formats out of scope:** encrypted mobile backups, tracev3 unified logs, `$LogFile` transaction replay, ext4 journal replay, APFS transparent compression, and XZ- or ZSTD-compressed journald payloads.
- **Acquisition on Linux and macOS** is implemented and unit-tested but has not run on real hosts.

- **Agreement is not correctness.** Where Veritas and a reference agree, both could be wrong. Every documented disagreement was resolved by a third source, but undiscovered shared errors remain possible, and the parity corpus, while public and varied, is small.
- **Cluster deployment** (container images, Terraform plan, deploy workflows) is validated statically; applying it requires CI credentials and a cloud account.

12. Conclusion

Veritas shows that the properties courts ask of forensic tools can be built in and measured rather than asserted. Evidence handling is a signed hash chain that anyone can verify offline. Parser correctness is a published, per-field agreement rate with every gap explained and a CI gate against regressions. Hostile input is contained by memory safety, fuzzing and process isolation. Reports are reproducible to the byte. On a public intrusion image the whole chain, from acquisition hash to verified custody export, runs unattended in about two minutes on a laptop.

Appendix A. Per-parser agreement

Generated from `parity/results/*.json`; field-level detail and notes are in `docs/PARITY.md`.

Parser	Reference	Fields	Agreeing	Ratio
evtx	libevtx-python 20260705	8	41,624 / 41,624	100.00%
ewf	libewf (pyewf) 20260924	7	48 / 48	100.00%
fs_apfs	pyfsapfs 20261030	20	146 / 146	100.00%
fs_ext4	pyfsext 20260821	13	543 / 543	100.00%
fs_ext4_tsk	pytsk3 4.15.0	8	368 / 368	100.00%
fs_fat	pyfsfat 20260826	10	98 / 140	70.00%
fs_fat_tsk	pytsk3 4.15.0	7	122 / 122	100.00%
fs_ntfs	pyfsntfs 20260827	15	197,649 / 197,649	100.00%
fs_ntfs_tsk	pytsk3 4.15.0	10	531 / 531	100.00%
fs_usn	dissect.ntfs 3.16 UsnJrnl	12	210 / 210	100.00%
fs_volume	pytsk3 4.15.0 Volume_Info (allocated partiti...	4	29 / 29	100.00%
fs_vss	pyvshadow 20260714	9	17 / 17	100.00%
journald	independent pure-Python journal reader (pyth...	4	13,034 / 13,199	98.75%
jumplist	libolecf-python 20260526 + liblnk-python 202...	15	126 / 126	100.00%
lnk	liblnk-python 20260525	21	63 / 63	100.00%
pcap	scapy 2.8.0	6	2,956 / 2,956	100.00%
pe_static	lief 1.0.0-d05b3499b	13	32 / 32	100.00%
plist	CPython 3.12.16 plistlib	2	772 / 772	100.00%
prefetch	libscca-python 20260527	9	83 / 83	100.00%
py_apache_access	plaso 20261027	2	28 / 30	93.33%
py_apt_history	plaso 20261027	2	30 / 30	100.00%

Parser	Reference	Fields	Agreeing	Ratio
py_asl_log	plaso 20261027	3	955 / 955	100.00%
py_auditd	raw text: distinct msg=audit(sec.ms:serial) ...	1	17 / 17	100.00%
py_bash_history	plaso 20261027	2	8 / 8	100.00%
py_bsm_log	plaso 20261027	1	104 / 104	100.00%
py_chromium_cookies	plaso 20261027	2	20 / 20	100.00%
py_chromium_history	plaso 20261027	3	194 / 194	100.00%
py_chromium_webdata	plaso 20261027	4	15 / 15	100.00%
py_dpkg	plaso 20261027	1	4 / 4	100.00%
py_firefox_cookies	plaso 20261027	2	52 / 52	100.00%
py_firefox_downloads	plaso 20261027	3	4 / 4	100.00%
py_firefox_places	plaso 20261027	2	519 / 523	99.24%
py_fish_history	plaso 20261027	2	20 / 20	100.00%
py_fseventsd	plaso 20261027	3	52 / 52	100.00%
py_imessage	plaso 20261027	2	20 / 20	100.00%
py_knowledgec	plaso 20261027	1	282 / 282	100.00%
py_launchd_plist	plaso 20261027	2	2 / 2	100.00%
py_ls_quarantine	plaso 20261027	3	42 / 42	100.00%
py_mac_appfirewall_log	plaso 20261027	2	93 / 94	98.94%
py_macos_appusage	plaso 20261027	2	10 / 10	100.00%
py_macos_background_items	plaso 20261027	1	1 / 1	100.00%
py_macos_launchd_log	plaso 20261027	1	36,609 / 36,609	100.00%
py_macos_notification_center	plaso 20261027	2	12 / 12	100.00%
py_macos_tcc	plaso 20261027	2	42 / 42	100.00%
py_macos_wifi	plaso 20261027	1	10 / 10	100.00%
py_mbox	python stdlib mailbox.mbox + email.utils	1	3 / 3	100.00%
py_msie_webcache	plaso 20261027	2	1,685 / 1,685	100.00%
py_msiecf	plaso 20261027	2	3,981 / 4,018	99.08%
py_olecf_summary	plaso 20261027	3	4 / 4	100.00%
py_ooxml	plaso 20261027	3	4 / 4	100.00%
py_powershell_transcript	plaso 20261027	1	1 / 1	100.00%
py_pst	libpff-python 20260926 (independent folder w...	2	1,084 / 1,084	100.00%
py_recycle_bin	plaso 20261027	2	4 / 4	100.00%
py_recycle_bin_info2	plaso 20261027	3	12 / 12	100.00%
py_santa	plaso 20261027	1	206 / 208	99.04%
py_setupapi	plaso 20261027	1	388 / 388	100.00%
py_srum	plaso 20261027	1	18,543 / 18,543	100.00%
py_syslog	plaso 20261027	1	42 / 43	97.67%
py_syslog_traditional	raw text (independent header regex), year-ag...	1	96 / 96	100.00%
py_utmp	plaso 20261027	2	30 / 30	100.00%
py_utmpx_macos	plaso 20261027	2	10 / 10	100.00%
py_windefender_history	plaso 20261027	4	12 / 12	100.00%

Parser	Reference	Fields	Agreeing	Ratio
py_windows_timeline	plaso 20261027	1	112 / 112	100.00%
py_wmi_persistence	hand-constructed OBJECTS.DATA byte fixture	1	4 / 4	100.00%
py_zsh_extended_history	plaso 20261027	2	8 / 8	100.00%
regf	libregf-python 20260526	7	1,148,381 / 1,156,890	99.26%
registry_artifacts	independent Python decoders over libregf-pyt...	13	1,852 / 1,852	100.00%
shellitems	libfwsi-python 20260522	9	405 / 405	100.00%

Appendix B. Test inventory

Suite	Count	What it covers
Rust unit, corpus and property tests	243	hashing, ledger and verifier, store, EWF, agent, file systems, artifacts, timeline
Python tests	713	API and RBAC, pipeline and sandbox, parsers, analytics, reporting, custody, store, cloud, mobile, memory, network, malware
Parity tests	68 result files	per-field agreement with reference implementations
Web unit tests	106	API client, keyboard handling, hex view, histogram, filters, role gating
Browser end-to-end tests	26	login to report download, two-person review, custody PASS, axe accessibility on every page
Fuzz targets	14	60-second smoke runs, about 5.5 million executions in total
Determinism samples	21	byte-identical parser output across runs (and across operating systems in CI)

Python line coverage is 91%; coverage of the web application's library and hooks is 98%.

Appendix C. Reproducing the results

```
python testdata/fetch.py                                  # reference corpus, SHA-256 pinned
cargo build --release --locked
pytest -m parity python/tests/parity && python scripts/parity_report.py
python scripts/determinism_check.py
python scripts/fuzz_smoke.py --seconds 60
python scripts/benchmark.py --disk-gib 2 --events 1000000
scripts/bootstrap/quickstart.sh --intrusion               # Hacking Case end to end
target/release/veritas-verify export var/bootstrap/<case>-custody.json
```

References

1 ISO/IEC 27037:2012, *Guidelines for identification, collection, acquisition and preservation of digital evidence*.

2 K. Kent, S. Chevalier, T. Grance, H. Dang, *NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response*, 2006.

3 Scientific Working Group on Digital Evidence, *Best Practices for Computer Forensic Acquisitions*.

4 Association of Chief Police Officers, *Good Practice Guide for Digital Evidence*, v5, 2012.

5 J. Metz, libyal format documentation (EVTX, REGF, SCCA, LNK, EWF, NTFS, ext, APFS, VSS).

6 B. Carrier, *File System Forensic Analysis*, Addison-Wesley, 2005; The Sleuth Kit.

7 Plaso (log2timeline) project and test data; dfVFS test data.

8 NIST Computer Forensic Reference Data Sets, *Hacking Case*.

9 Microsoft, [MS-SHLLINK], [MS-XCA], [MS-EVEN6] open specifications.

10 A. Walters et al., The Volatility Framework, version 3.

11 A. Rundgren, B. Jordan, S. Erdtman, RFC 8785, *JSON Canonicalization Scheme*, 2020.

12 D. J. Bernstein et al., *High-speed high-security signatures* (Ed25519), 2012.